This example will include the APIHelpers module into *every* example group. Most of the time, you probably want to load this module into just the groups that need it. In *Sharing Code Conditionally*, on page 140, we'll show you how.

As with all global settings that are buried away in a support file, take care not to hide important details behind a call to config.include.

Sharing Example Groups

As we've seen, plain old Ruby modules work really nicely for sharing helper methods across example groups. But that's all they can share. If you want to reuse an example, a let construct or a hook, you'll need to reach for another tool: *shared example groups*.

Just like its non-shared counterpart, a shared example group can contain examples, helper methods, let declarations, and hooks. The only difference is the way they're used. A shared example group exists *only* to be shared.

In order to help you "get the words right," RSpec provides multiple ways to create and use shared example groups. These come in pairs, with one method for *defining* a shared group and another for *using* it:

- shared_context and include_context are for reusing *common setup* and *helper logic*.
- shared_examples and include_examples are for reusing *examples*.

The choice between the ..._context and ..._examples wording is purely a matter of communicating your intent. Behind the scenes, they behave identically.

There's one more way to share behavior that *is* different, though. it_behaves_like creates a new, *nested* example group to hold the shared code. The difference lies in how isolated the shared behavior is from the rest of your examples. In the coming sections, we'll talk about when you'd want to use each approach.

Sharing Contexts

Earlier in this chapter, we saw that you can group common helper methods into a module:

07-structuring-code-examples/08/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
module APIHelpers
  include Rack::Test::Methods

  def app
    ExpenseTracker::API.new
  end
end
```